

SQL WORKSHOP - TRAINING ACADEMY

Microsoft SQL Server / T-SQL

Topics:

- Database and table creation
- Primary Keys and Foreign Keys
- INSERT
- SELECT
- WHERE
- ORDER BY
- JOIN
- GROUP BY
- CASE
- Attendance analysis
- Exams
- HAVING

Run the script section by section during the workshop.

```
-- =====
```

```
-- 1. CREATE DATABASE
```

```
-- =====
```

```
CREATE DATABASE TrainingAcademy;
```

```
USE TrainingAcademy;
```

```
-- =====
```

-- 2. CREATE STUDENTS TABLE

-- =====

```
CREATE TABLE Students (  
  StudentID INT PRIMARY KEY,  
  FullName VARCHAR(100) NOT NULL,  
  Email VARCHAR(100) UNIQUE,  
  BirthDate DATE  
);
```

-- =====

-- 3. CREATE INSTRUCTORS TABLE

-- =====

```
CREATE TABLE Instructors (  
  InstructorID INT PRIMARY KEY,  
  FullName VARCHAR(100) NOT NULL,  
  Specialization VARCHAR(100)  
);
```

-- =====

-- 4. CREATE COURSES TABLE

-- Each course belongs to one instructor.

-- =====

```
CREATE TABLE Courses (  
  CourseID INT PRIMARY KEY,
```

```
CourseName VARCHAR(100) NOT NULL,  
Fee DECIMAL(10,2),  
InstructorID INT,  
FOREIGN KEY (InstructorID)  
REFERENCES Instructors(InstructorID)  
);
```

```
-- =====
```

```
-- 5. CREATE ENROLLMENTS TABLE
```

```
-- Connects students with courses.
```

```
-- =====
```

```
CREATE TABLE Enrollments (  
EnrollmentID INT PRIMARY KEY,  
StudentID INT,  
CourseID INT,  
EnrollmentDate DATE,  
FOREIGN KEY (StudentID)  
REFERENCES Students(StudentID),  
FOREIGN KEY (CourseID)  
REFERENCES Courses(CourseID)  
);
```

```
-- =====
```

```
-- 6. CREATE SESSIONS TABLE
```

```
-- =====
```

```
CREATE TABLE Sessions (  
SessionID INT PRIMARY KEY,  
CourseID INT,  
SessionDate DATE,  
Topic VARCHAR(200),  
FOREIGN KEY (CourseID)  
REFERENCES Courses(CourseID)  
);
```

```
-- =====  
-- 7. CREATE ATTENDANCE TABLE  
-- =====
```

```
CREATE TABLE Attendance (  
AttendanceID INT PRIMARY KEY,  
SessionID INT,  
StudentID INT,  
Status VARCHAR(20),  
FOREIGN KEY (SessionID)  
REFERENCES Sessions(SessionID),  
FOREIGN KEY (StudentID)  
REFERENCES Students(StudentID)  
);
```

```
-- =====  
-- 8. INSERT STUDENTS  
-- =====
```

INSERT INTO Students

(StudentID, FullName, Email, BirthDate)

VALUES

(1, 'Ahmad Ali', 'ahmad@email.com', '2001-05-12'),
(2, 'Sara Khaled', 'sara@email.com', '2002-08-20'),
(3, 'Omar Hassan', 'omar@email.com', '2000-11-03'),
(4, 'Lina Samir', 'lina@email.com', '2003-02-15');

-- =====

-- 9. INSERT INSTRUCTORS

-- =====

INSERT INTO Instructors

(InstructorID, FullName, Specialization)

VALUES

(1, 'Mohammad Ahmad', 'Database Development'),
(2, 'Sara Omar', 'Web Development');

-- =====

-- 10. INSERT COURSES

-- =====

INSERT INTO Courses

(CourseID, CourseName, Fee, InstructorID)

VALUES

(1, 'SQL Fundamentals', 120.00, 1),

```
(2, 'Web Development', 180.00, 2),  
(3, 'Advanced SQL', 200.00, 1);
```

```
-- =====
```

```
-- 11. INSERT ENROLLMENTS
```

```
-- =====
```

```
INSERT INTO Enrollments
```

```
(EnrollmentID, StudentID, CourseID, EnrollmentDate)
```

```
VALUES
```

```
(1, 1, 1, '2026-09-01'),  
(2, 2, 1, '2026-09-01'),  
(3, 3, 1, '2026-09-02'),  
(4, 1, 2, '2026-09-03'),  
(5, 4, 3, '2026-09-04');
```

```
-- =====
```

```
-- 12. INSERT SESSIONS
```

```
-- =====
```

```
INSERT INTO Sessions
```

```
(SessionID, CourseID, SessionDate, Topic)
```

```
VALUES
```

```
(1, 1, '2026-09-10', 'Introduction to SQL'),  
(2, 1, '2026-09-12', 'SELECT and WHERE'),  
(3, 1, '2026-09-15', 'JOINS'),  
(4, 2, '2026-09-11', 'HTML Basics'),
```

```
(5, 3, '2026-09-16', 'Advanced Queries');
```

```
-- =====
```

```
-- 13. INSERT ATTENDANCE
```

```
-- =====
```

```
INSERT INTO Attendance
```

```
(AttendanceID, SessionID, StudentID, Status)
```

```
VALUES
```

```
(1, 1, 1, 'Present'),
```

```
(2, 1, 2, 'Present'),
```

```
(3, 1, 3, 'Absent'),
```

```
(4, 2, 1, 'Present'),
```

```
(5, 2, 2, 'Absent'),
```

```
(6, 2, 3, 'Present'),
```

```
(7, 3, 1, 'Present'),
```

```
(8, 3, 2, 'Present'),
```

```
(9, 3, 3, 'Present');
```

```
-- =====
```

```
-- 14. SELECT - ALL STUDENTS
```

```
-- =====
```

```
SELECT *
```

```
FROM Students;
```

```
-- =====
```

-- 15. SELECT - SPECIFIC COLUMNS

-- =====

SELECT FullName, Email

FROM Students;

-- =====

-- 16. WHERE - STUDENTS BORN AFTER 2001

-- =====

SELECT *

FROM Students

WHERE BirthDate > '2001-01-01';

-- =====

-- 17. WHERE - COURSES WITH FEE GREATER THAN 150

-- =====

SELECT *

FROM Courses

WHERE Fee > 150;

-- =====

-- 18. ORDER BY - MOST EXPENSIVE FIRST

-- =====

```
SELECT *  
FROM Courses  
ORDER BY Fee DESC;
```

```
-- =====  
-- 19. JOIN - COURSES WITH THEIR INSTRUCTORS  
-- =====
```

```
SELECT  
Courses.CourseName,  
Courses.Fee,  
Instructors.FullName AS Instructor  
FROM Courses  
JOIN Instructors  
ON Courses.InstructorID = Instructors.InstructorID;
```

```
-- =====  
-- 20. JOIN - STUDENTS AND THEIR COURSES  
-- =====
```

```
SELECT  
Students.FullName AS Student,  
Courses.CourseName AS Course  
FROM Enrollments  
JOIN Students  
ON Enrollments.StudentID = Students.StudentID  
JOIN Courses
```

ON Enrollments.CourseID = Courses.CourseID;

-- =====

-- 21. GROUP BY - STUDENTS PER COURSE

-- =====

```
SELECT
CourseID,
COUNT(*) AS NumberOfStudents
FROM Enrollments
GROUP BY CourseID;
```

-- =====

-- 22. GROUP BY + JOIN - COURSE NAMES AND STUDENT COUNTS

-- =====

```
SELECT
Courses.CourseName,
COUNT(Enrollments.StudentID) AS NumberOfStudents
FROM Courses
JOIN Enrollments
ON Courses.CourseID = Enrollments.CourseID
GROUP BY Courses.CourseName;
```

-- =====

-- 23. SHOW ALL ATTENDANCE

-- =====

```
SELECT *  
FROM Attendance;
```

-- =====

-- 24. SHOW ABSENT STUDENTS

-- =====

```
SELECT *  
FROM Attendance  
WHERE Status = 'Absent';
```

-- =====

-- 25. ATTENDANCE WITH STUDENT NAMES

-- =====

```
SELECT  
Students.FullName,  
Attendance.Status  
FROM Attendance  
JOIN Students  
ON Attendance.StudentID = Students.StudentID;
```

-- =====

-- 26. COMPLETE ATTENDANCE DETAILS

-- =====

```
SELECT
Students.FullName AS Student,
Sessions.SessionDate,
Sessions.Topic,
Attendance.Status
FROM Attendance
JOIN Students
ON Attendance.StudentID = Students.StudentID
JOIN Sessions
ON Attendance.SessionID = Sessions.SessionID;
```

-- =====

-- 27. BUSINESS QUESTION:
-- How many sessions did each student attend?

-- =====

```
SELECT
Students.FullName,
COUNT(*) AS PresentSessions
FROM Attendance
JOIN Students
ON Attendance.StudentID = Students.StudentID
WHERE Attendance.Status = 'Present'
GROUP BY Students.FullName;
```

```
-- =====  
-- 28. ATTENDANCE SUMMARY  
-- =====
```

```
SELECT  
Students.FullName,  
COUNT(*) AS TotalSessions,  
SUM(  
CASE  
WHEN Attendance.Status = 'Present' THEN 1  
ELSE 0  
END  
) AS PresentSessions  
FROM Attendance  
JOIN Students  
ON Attendance.StudentID = Students.StudentID  
GROUP BY Students.FullName;
```

```
-- =====  
-- 29. ATTENDANCE PERCENTAGE  
-- =====
```

```
SELECT  
Students.FullName,  
COUNT(*) AS TotalSessions,  
SUM(  
CASE  
WHEN Attendance.Status = 'Present' THEN 1
```

```
ELSE 0
END
) AS PresentSessions,
CAST(
SUM(
CASE
WHEN Attendance.Status = 'Present' THEN 1
ELSE 0
END
) * 100.0 / COUNT(*)
AS DECIMAL(5,2)
) AS AttendancePercentage
FROM Attendance
JOIN Students
ON Attendance.StudentID = Students.StudentID
GROUP BY Students.FullName;
```